

Analytical Window Functions: LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE

Table of Contents

- 1. [Learning Objectives](#)
 - 2. [Theory: Navigation Functions](#)
 - 3. [LAG](#)
 - 4. [LEAD](#)
 - 5. [FIRST_VALUE](#)
 - 6. [LAST_VALUE](#)
 - 7. [NTH_VALUE](#)
 - 8. [PERCENT_RANK and CUME_DIST](#)
 - 9. [Combining Analytical Functions](#)
 - 10. [ASCII Visual Diagrams](#)
 - 11. [Real-World Use Cases](#)
 - 12. [Common Mistakes](#)
 - 13. [Best Practices](#)
 - 14. [Performance Considerations](#)
 - 15. [Interview Questions & Answers](#)
 - 16. [Hands-on Exercises](#)
 - 17. [Advanced Notes](#)
 - 18. [Cross-references](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Use LAG and LEAD to compare a row to its predecessor/successor
 - Apply FIRST_VALUE and LAST_VALUE to access boundary values of a partition
 - Use NTH_VALUE to access the Nth row in a window
 - Build period-over-period comparisons and trend analysis
 - Solve common analytical interview problems
-

Theory: Navigation Functions

Navigation (offset) window functions access values from other rows within the same window without a self-join. They are essential for time-series analysis, trend detection, and period-over-period comparisons.

Row position relative to current row:

↑ offset rows before

[R1][R2][R3][R4][R5][R6][R7][R8][R9]

↑

CURRENT

↓ offset rows after

LAG(val, N)

= value from N rows BEFORE current

LEAD(val, N)

= value from N rows AFTER current

FIRST_VALUE

= value from the FIRST row in the window frame

LAST_VALUE	= value from the LAST row in the window frame
NTH_VALUE(N)	= value from the Nth row in the window frame

LAG

LAG accesses values from preceding rows within the same partition. The default offset is 1 (immediately preceding row).

Syntax

```
LAG(expression [, offset [, default]])  
  OVER ([PARTITION BY ...] ORDER BY ...)
```

- `offset` : how many rows back (default 1)
- `default` : value to return when no preceding row exists (default NULL)

Examples

```
-- Example 1: Month-over-month revenue comparison  
SELECT  
    department,  
    month_date,  
    revenue,  
    LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS  
prev_month_rev,  
    revenue - LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS  
mom_change,  
    ROUND(100.0 * (revenue - LAG(revenue) OVER (PARTITION BY department ORDER BY  
month_date))  
        / NULLIF(LAG(revenue) OVER (PARTITION BY department ORDER BY month_date),  
0), 2  
    ) AS mom_pct_change  
FROM monthly_revenue  
ORDER BY department, month_date;  
  
-- Example 2: LAG with offset of 2 – compare to 2 months ago  
SELECT  
    department,  
    month_date,  
    revenue,  
    LAG(revenue, 2, 0) OVER (PARTITION BY department ORDER BY month_date) AS  
revenue_2m_ago,  
    revenue - LAG(revenue, 2, 0) OVER (PARTITION BY department ORDER BY month_date)  
AS change_vs_2m  
FROM monthly_revenue  
ORDER BY department, month_date;  
  
-- Example 3: LAG with default value for first row  
SELECT  
    department,
```

```

    month_date,
    revenue,
    LAG(revenue, 1, revenue) OVER (PARTITION BY department ORDER BY month_date) AS
prev_or_self,
    -- prev_or_self is same as revenue for the first row
    revenue - LAG(revenue, 1, revenue) OVER (PARTITION BY department ORDER BY
month_date) AS change
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 4: Compare employee salary to previous employee hired
SELECT
    emp_name,
    department,
    hire_date,
    salary,
    LAG(salary) OVER (PARTITION BY department ORDER BY hire_date) AS
prev_hire_salary,
    salary - LAG(salary) OVER (PARTITION BY department ORDER BY hire_date) AS
salary_vs_prev_hire
FROM emp_salary
ORDER BY department, hire_date;

-- Example 5: Detect consecutive monthly declines
SELECT
    department,
    month_date,
    revenue,
    LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS prev_revenue,
    CASE WHEN revenue < LAG(revenue) OVER (PARTITION BY department ORDER BY
month_date)
        THEN 'Decline'
        WHEN revenue > LAG(revenue) OVER (PARTITION BY department ORDER BY
month_date)
        THEN 'Growth'
        ELSE 'Flat / First Month'
    END AS trend
FROM monthly_revenue
ORDER BY department, month_date;

```

LEAD

LEAD accesses values from following (future) rows within the same partition.

Syntax

```

LEAD(expression [, offset [, default]])
    OVER ([PARTITION BY ...] ORDER BY ...)

```

Examples

```
-- Example 6: Show next month's revenue alongside current
SELECT
    department,
    month_date,
    revenue,
    LEAD(revenue) OVER (PARTITION BY department ORDER BY month_date) AS
next_month_rev,
    LEAD(revenue) OVER (PARTITION BY department ORDER BY month_date) - revenue AS
next_vs_current
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 7: Time to next hire per department
SELECT
    emp_name,
    department,
    hire_date,
    LEAD(hire_date) OVER (PARTITION BY department ORDER BY hire_date) AS
next_hire_date,
    LEAD(hire_date) OVER (PARTITION BY department ORDER BY hire_date) - hire_date AS
days_between_hires
FROM emp_salary
ORDER BY department, hire_date;

-- Example 8: LEAD with default – last row gets a default
SELECT
    department,
    month_date,
    revenue,
    LEAD(revenue, 1, 0) OVER (PARTITION BY department ORDER BY month_date) AS
next_rev_or_zero
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 9: Detect when a streak ends
SELECT
    department,
    month_date,
    revenue,
    LEAD(revenue) OVER (PARTITION BY department ORDER BY month_date) AS next_rev,
    CASE
        WHEN revenue > LEAD(revenue) OVER (PARTITION BY department ORDER BY
month_date)
        THEN 'Peak this month'
        ELSE 'Not a peak'
    END AS is_local_peak
FROM monthly_revenue
ORDER BY department, month_date;
```

```
-- Example 10: Both LAG and LEAD – show context window
SELECT
    department,
    month_date,
    LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS prev_rev,
    revenue AS curr_rev,
    LEAD(revenue) OVER (PARTITION BY department ORDER BY month_date) AS next_rev
FROM monthly_revenue
ORDER BY department, month_date;
```

FIRST_VALUE

Returns the value of an expression evaluated at the first row of the window frame.

Syntax

```
FIRST_VALUE(expression) OVER (
    [PARTITION BY ...]
    ORDER BY ...
    [frame_clause]
)
```

Important Note

Without an explicit frame clause, FIRST_VALUE uses RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. This always returns the same first row of the partition (since UNBOUNDED PRECEDING is the start). But specifying ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING makes it truly the first row of the entire partition.

Examples

```
-- Example 11: First (oldest) hire's salary in each department
SELECT
    emp_name,
    department,
    hire_date,
    salary,
    FIRST_VALUE(emp_name) OVER (
        PARTITION BY department
        ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS first_hire,
    FIRST_VALUE(salary) OVER (
        PARTITION BY department
        ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS first_hire_salary
FROM emp_salary
ORDER BY department, hire_date;
```

```

-- Example 12: First month's revenue as a baseline
SELECT
    department,
    month_date,
    revenue,
    FIRST_VALUE(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS jan_baseline,
    ROUND(100.0 * revenue /
        FIRST_VALUE(revenue) OVER (
            PARTITION BY department
            ORDER BY month_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        ), 2) AS pct_of_jan
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 13: Compare each sale to the best sale in the department
SELECT
    emp_name,
    department,
    salary,
    FIRST_VALUE(salary) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS dept_top_salary,
    ROUND(100.0 * salary /
        FIRST_VALUE(salary) OVER (
            PARTITION BY department
            ORDER BY salary DESC
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        ), 2) AS pct_of_top
FROM emp_salary
ORDER BY department, salary DESC;

```

LAST_VALUE

Returns the value from the last row of the window frame.

Critical Gotcha with LAST_VALUE

The default frame is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. So LAST_VALUE with default frame returns the **current row** value — which is useless! Always specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING to get the true last value.

Examples

```

-- Example 14: LAST_VALUE – CORRECT usage with explicit frame
SELECT
    department,
    month_date,
    revenue,
    LAST_VALUE(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_month_revenue,
    LAST_VALUE(month_date) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_month_date
FROM monthly_revenue
ORDER BY department, month_date;

-- Example 15: LAST_VALUE vs LAG alternative (both valid)
SELECT
    emp_name,
    department,
    hire_date,
    salary,
    -- LAST_VALUE to get most recently hired person's salary
    LAST_VALUE(emp_name) OVER (
        PARTITION BY department
        ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS latest_hire
FROM emp_salary
ORDER BY department, hire_date;

-- Example 16: WRONG usage of LAST_VALUE (returns current row = useless)
SELECT
    department,
    month_date,
    revenue,
    LAST_VALUE(revenue) OVER ( -- WRONG: default frame makes this = revenue
        PARTITION BY department
        ORDER BY month_date
    ) AS wrong_last_value
FROM monthly_revenue;

```

NTH_VALUE

Returns the value from the Nth row of the window frame (1-based indexing).

Examples

```
-- Example 17: NTH_VALUE – 2nd highest salary in department
SELECT
    emp_name,
    department,
    salary,
    NTH_VALUE(salary, 1) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS highest_salary,
    NTH_VALUE(salary, 2) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS second_highest_salary,
    NTH_VALUE(salary, 3) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS third_highest_salary
FROM emp_salary
ORDER BY department, salary DESC;
```

```
-- Example 18: NTH_VALUE for revenue milestones
SELECT
    department,
    month_date,
    revenue,
    NTH_VALUE(revenue, 2) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS feb_revenue
FROM monthly_revenue
ORDER BY department, month_date;
```

PERCENT_RANK and CUME_DIST

```
-- Example 19: PERCENT_RANK and CUME_DIST
SELECT
    emp_name,
    salary,
    department,
    ROUND(PERCENT_RANK() OVER (ORDER BY salary), 4) AS pct_rank_global,
    ROUND(CUME_DIST() OVER (ORDER BY salary), 4) AS cume_dist_global,
    ROUND(PERCENT_RANK() OVER (PARTITION BY department ORDER BY salary), 4) AS
pct_rank_dept,
    ROUND(CUME_DIST() OVER (PARTITION BY department ORDER BY salary), 4) AS
cume_dist_dept
```



```

FROM emp_salary
ORDER BY salary;
-- PERCENT_RANK: (rank-1)/(rows-1), range [0, 1]
-- CUME_DIST:     (rank)/(rows), range (0, 1]

```

Combining Analytical Functions

```

-- Example 20: Full analytical report – trend, comparison, context
SELECT
    department,
    month_date,
    revenue,
    -- Previous and next month
    LAG(revenue) OVER w AS prev_month,
    LEAD(revenue) OVER w AS next_month,
    -- Change calculations
    revenue - LAG(revenue) OVER w AS mom_change,
    ROUND(100.0 * (revenue - LAG(revenue) OVER w)
          / NULLIF(LAG(revenue) OVER w, 0), 2) AS mom_pct,
    -- Boundaries
    FIRST_VALUE(revenue) OVER (PARTITION BY department
                              ORDER BY month_date
                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS jan_rev,
    LAST_VALUE(revenue) OVER (PARTITION BY department
                              ORDER BY month_date
                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS dec_rev,
    -- Running total
    SUM(revenue) OVER (PARTITION BY department ORDER BY month_date
                      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS ytd_revenue
FROM monthly_revenue
WINDOW w AS (PARTITION BY department ORDER BY month_date)
ORDER BY department, month_date;

```

ASCII Visual Diagrams

LAG and LEAD Visualization

Monthly Revenue (Engineering): Jan=45000, Feb=52000, Mar=48000, Apr=61000

Month	Revenue	LAG(1)	LEAD(1)	LAG(2)
Jan	45000	NULL	52000	NULL
Feb	52000	45000	48000	NULL
Mar	48000	52000	61000	45000
Apr	61000	48000	NULL	52000
	↑	↑	↑	
	current	prev	next	

FIRST_VALUE and LAST_VALUE with Frame

Partition: Engineering months ordered by date

Default frame (RANGE UNBOUNDED PRECEDING TO CURRENT ROW):

Jan: FIRST_VALUE=Jan, LAST_VALUE=Jan ← only Jan in frame
Feb: FIRST_VALUE=Jan, LAST_VALUE=Feb ← Jan through Feb
Mar: FIRST_VALUE=Jan, LAST_VALUE=Mar ← Jan through Mar
Apr: FIRST_VALUE=Jan, LAST_VALUE=Apr ← Jan through Apr

↑
LAST_VALUE = current row! (not useful)

Full frame (ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING):

Jan: FIRST_VALUE=Jan, LAST_VALUE=Apr ← all 4 months
Feb: FIRST_VALUE=Jan, LAST_VALUE=Apr ← all 4 months
Mar: FIRST_VALUE=Jan, LAST_VALUE=Apr ← all 4 months
Apr: FIRST_VALUE=Jan, LAST_VALUE=Apr ← all 4 months

↑
LAST_VALUE = truly last row!

Real-World Use Cases

Year-over-Year Comparison

```
-- Year-over-year using LAG (offset = 12 for monthly data)
SELECT
    department,
    month_date,
    revenue,
    LAG(revenue, 12) OVER (PARTITION BY department ORDER BY month_date) AS
same_month_last_year,
    ROUND(100.0 * (revenue - LAG(revenue, 12) OVER (PARTITION BY department ORDER BY
month_date))
        / NULLIF(LAG(revenue, 12) OVER (PARTITION BY department ORDER BY
month_date), 0), 2
    ) AS yoy_pct_change
FROM monthly_revenue
ORDER BY department, month_date;
```

Consecutive Condition Detection (Streak)

```
-- Find rows where revenue grew for 2 consecutive months
SELECT department, month_date, revenue
FROM (
    SELECT
        department,
        month_date,
        revenue,
```

```
LAG(revenue, 1) OVER (PARTITION BY department ORDER BY month_date) AS m1,  
LAG(revenue, 2) OVER (PARTITION BY department ORDER BY month_date) AS m2  
FROM monthly_revenue  
) AS t  
WHERE revenue > m1 AND m1 > m2;
```

Common Mistakes

Mistake 1: LAST_VALUE returning current row

```
-- WRONG: default frame makes LAST_VALUE = current row  
LAST_VALUE(salary) OVER (PARTITION BY dept ORDER BY salary)  
  
-- CORRECT:  
LAST_VALUE(salary) OVER (PARTITION BY dept ORDER BY salary  
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

Mistake 2: NTH_VALUE indexing starts at 1, not 0

```
-- NTH_VALUE(col, 1) = first value (same as FIRST_VALUE)  
-- NTH_VALUE(col, 0) → ERROR: N must be >= 1
```

Mistake 3: LAG/LEAD returning NULL without default

```
-- First row has no predecessor → LAG returns NULL  
-- Use COALESCE or default parameter:  
LAG(revenue, 1, 0) OVER (...) -- default 0  
COALESCE(LAG(revenue) OVER (...), 0) -- same result
```

Mistake 4: Wrong PARTITION BY — LAG crossing group boundaries

```
-- If you forget PARTITION BY, LAG will look at the previous row  
-- regardless of which department or group it belongs to!  
LAG(salary) OVER (ORDER BY department, salary) -- crosses dept boundary!  
  
-- CORRECT:  
LAG(salary) OVER (PARTITION BY department ORDER BY salary)
```

Best Practices

1. Always specify an explicit frame clause when using FIRST_VALUE, LAST_VALUE, NTH_VALUE.
 2. Always include PARTITION BY with LAG/LEAD unless you intentionally want cross-group comparison.
 3. Use the default parameter in LAG/LEAD to avoid NULL for edge rows.
 4. Use NULLIF in denominators for percentage calculations with LAG.
 5. Combine LAG/LEAD in a single subquery to avoid rescanning the table multiple times.
-

Performance Considerations

```
-- Multiple analytical functions in one subquery = one table scan
-- vs multiple subqueries = multiple table scans

-- GOOD: single scan
SELECT *, LAG(x) OVER w AS lag_x, LEAD(x) OVER w AS lead_x
FROM t
WINDOW w AS (PARTITION BY grp ORDER BY dt);

-- BAD: two separate scans
SELECT *, (SELECT x FROM t t2 WHERE ... ORDER BY dt DESC LIMIT 1) AS lag_x FROM t;
SELECT *, (SELECT x FROM t t2 WHERE ... ORDER BY dt ASC LIMIT 1) AS lead_x FROM t;

-- Index on PARTITION BY + ORDER BY columns:
CREATE INDEX idx_rev_dept_month ON monthly_revenue(department, month_date);
```

Interview Questions & Answers

Q1: What is the difference between LAG and LEAD?

A: LAG accesses a value from a preceding row (offset rows back), while LEAD accesses a value from a following row (offset rows forward). Both accept an optional offset (default 1) and a default value for edge rows. They are commonly used for period-over-period comparisons.

Q2: What is the LAST_VALUE gotcha?

A: The default frame for window functions with ORDER BY is RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. With this frame, LAST_VALUE returns the current row's value (the last row seen so far). To get the true last row of the partition, you must explicitly specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

Q3: What does LAG return for the first row in a partition?

A: NULL by default (no preceding row exists). You can provide a third parameter to LAG as the default value: `LAG(revenue, 1, 0)` returns 0 for the first row.

Q4: How would you compute month-over-month growth rate using LAG?

A:

```
ROUND(100.0 * (revenue - LAG(revenue) OVER (PARTITION BY dept ORDER BY month_date))
      / NULLIF(LAG(revenue) OVER (PARTITION BY dept ORDER BY month_date), 0), 2) AS
mom_pct
```

Use NULLIF to prevent division by zero.

Q5: What is NTH_VALUE and when would you use it?

A: NTH_VALUE(expression, N) returns the value from the Nth row of the window frame. Use it when you need the 2nd, 3rd, or any specific positional value from a group without multiple subqueries. Requires ROWS

BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING for full-partition access.

Q6: Can LAG access values 3 rows back?

A: Yes. `LAG(salary, 3)` accesses the salary from 3 rows back. The offset can be any positive integer expression.

Q7: How is FIRST_VALUE different from MIN()?

A: `MIN()` returns the minimum value in the group. `FIRST_VALUE()` returns the value from the first row of the window frame based on the `ORDER BY`, which may not be the minimum. If you `ORDER BY salary DESC`, `FIRST_VALUE(salary) = MAX salary`, not `MIN`.

Q8: How would you find the first and last sale date for each customer?

A:

```
SELECT DISTINCT
  customer_id,
  FIRST_VALUE(order_date) OVER w AS first_order,
  LAST_VALUE(order_date) OVER w AS last_order
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date
              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

Or simpler: `MIN(order_date)` and `MAX(order_date)` with `GROUP BY`.

Hands-on Exercises

Exercise 1: MoM Revenue Change

For the `monthly_revenue` table, show each row alongside its previous month's revenue and the month-over-month change and percentage.

```
-- Solution:
SELECT
  department,
  month_date,
  revenue,
  LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS prev_rev,
  revenue - LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS
change,
  ROUND(100.0 * (revenue - LAG(revenue) OVER (PARTITION BY department ORDER BY
month_date))
        / NULLIF(LAG(revenue) OVER (PARTITION BY department ORDER BY month_date),
0), 2) AS pct_chg
FROM monthly_revenue
ORDER BY department, month_date;
```

Exercise 2: First vs Latest Salary in Department

For each employee, show the salary of the first person hired in their department and the most recently hired person.

```
-- Solution:
SELECT
    emp_name,
    department,
    hire_date,
    salary,
    FIRST_VALUE(salary) OVER (
        PARTITION BY department ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS first_hired_salary,
    LAST_VALUE(salary) OVER (
        PARTITION BY department ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS last_hired_salary
FROM emp_salary
ORDER BY department, hire_date;
```

Exercise 3: Revenue Peaks

Identify months where revenue was higher than both the previous and next months (local peaks).

```
-- Solution:
SELECT department, month_date, revenue
FROM (
    SELECT
        department,
        month_date,
        revenue,
        LAG(revenue) OVER (PARTITION BY department ORDER BY month_date) AS
prev_rev,
        LEAD(revenue) OVER (PARTITION BY department ORDER BY month_date) AS next_rev
    FROM monthly_revenue
) AS t
WHERE revenue > COALESCE(prev_rev, 0)
    AND revenue > COALESCE(next_rev, 0)
ORDER BY department, month_date;
```

Exercise 4: Salary Journey

Show the 2nd highest salary in each department using NTH_VALUE.

```
-- Solution:
SELECT DISTINCT
    department,
    NTH_VALUE(salary, 2) OVER (
        PARTITION BY department
        ORDER BY salary DESC
```

```
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS second_highest_salary
FROM emp_salary
ORDER BY department;
```

Advanced Notes

LAG/LEAD over Aggregated Results

```
-- You can use LAG/LEAD on the output of a GROUP BY
SELECT
    department,
    yr,
    annual_rev,
    LAG(annual_rev) OVER (PARTITION BY department ORDER BY yr) AS prev_year_rev,
    annual_rev - LAG(annual_rev) OVER (PARTITION BY department ORDER BY yr) AS
yoy_change
FROM (
    SELECT
        department,
        EXTRACT(YEAR FROM month_date) AS yr,
        SUM(revenue) AS annual_rev
    FROM monthly_revenue
    GROUP BY department, EXTRACT(YEAR FROM month_date)
) AS yearly
ORDER BY department, yr;
```

Cross-references

- [01_window_functions_intro.md](#) — OVER, PARTITION BY, frame clauses foundation
- [02_ranking_functions.md](#) — ROW_NUMBER, RANK, DENSE_RANK for positional access
- [08_interview_window_tasks.md](#) — 20+ company interview problems
- [04_ctes.md](#) — CTEs to stage LAG/LEAD results before further filtering